

Project Title

Personal Expense Splitter System Using
DBMS

By

Srishty Agarwal

2410030651

1. Problem Statement

-> Managing shared expenses among groups (friends, roommates, trips) is difficult. It becomes confusing to track who paid, who owes money, and how to settle balances.

This project provides a database system that:

- Stores users and group details
- Tracks expenses
- Splits expenses among users
- Maintains settlement records

2. Database Schema (Tables)

✓ Users Table

- UserID (PK)
- Name
- Email
- PasswordHash
- CreatedAt

✓ Groups Table

- GroupID (PK)
- GroupName
- CreatedBy (FK)

✓ GroupMembers Table

- GroupID (FK)
- UserID (FK)
- Role

✓ Expenses Table

- ExpenseID (PK)
- GroupID (FK)
- PaidBy (FK)
- TotalAmount

- ExpenseDate

✓ ExpenseSplits Table

- SplitID (PK)
- ExpenseID (FK)
- UserID (FK)
- OwedAmount
- PaidAmount

✓ Settlements Table

- SettlementID (PK)
- GroupID (FK)
- PaidBy (FK)
- PaidTo (FK)
- Amount
- SettlementDate

3. SQL QUERIES WITH OUTPUT

Query 1: SELECT with Condition

Problem: Find all users using Gmail.

Query: SELECT * FROM Users

WHERE Email LIKE '%gmail%';

```
MariaDB [ExpenseSplitter]> SELECT * FROM Users
-> WHERE Email LIKE '%gmail%';
```

UserID	Name	Email	PasswordHash	CreatedAt
1	shreya	shreya@gmail.com	pass123	2026-03-29 23:10:12
2	Rahul	rahul@gmail.com	pass123	2026-03-29 23:10:12
3	Aman	aman@gmail.com	pass123	2026-03-29 23:10:12
4	Priya	priya@gmail.com	pass123	2026-03-29 23:10:12

```
rows in set (0.001 sec)
```

Query 2: SELECT with Condition

Problem: Find expenses greater than 1000.

Query: SELECT * FROM Expenses

WHERE TotalAmount > 1000;

```
MariaDB [ExpenseSplitter]> SELECT * FROM Expenses
-> WHERE TotalAmount > 1000;
```

ExpenseID	GroupID	PaidBy	TotalAmount	Title	ExpenseDate
1	1	1	1200.00	Dinner	2026-03-20
3	2	2	3000.00	Rent	2026-03-01

2 rows in set (0.001 sec)

Query 3: JOIN

Problem: Display expense details with group name and payer name.

Query: SELECT e.Title, g.GroupName, u.Name AS PaidBy, e.TotalAmount

FROM Expenses e

JOIN Groups g ON e.GroupID = g.GroupID

JOIN Users u ON e.PaidBy = u.UserID;

```
MariaDB [ExpenseSplitter]> SELECT e.Title, g.GroupName, u.Name AS PaidBy, e.TotalAmount
-> FROM Expenses e
-> JOIN Groups g ON e.GroupID = g.GroupID
-> JOIN Users u ON e.PaidBy = u.UserID;
```

Title	GroupName	PaidBy	TotalAmount
Dinner	Trip Group	shreya	1200.00
Taxi	Trip Group	Rahul	800.00
Rent	Flat Expenses	Rahul	3000.00

3 rows in set (0.001 sec)

Query 4: JOIN with Condition

Problem: Show all members of Trip Group.

SELECT u.Name, gm.Role

FROM GroupMembers gm

JOIN Users u ON gm.UserID = u.UserID

JOIN Groups g ON gm.GroupID = g.GroupID

WHERE g.GroupName = 'Trip Group';

```

MariaDB [ExpenseSplitter]> SELECT u.Name, gm.Role
-> FROM GroupMembers gm
-> JOIN Users u ON gm.UserID = u.UserID
-> JOIN Groups g ON gm.GroupID = g.GroupID
-> WHERE g.GroupName = 'Trip Group';
+-----+-----+
| Name | Role |
+-----+-----+
| shreya | Admin |
| Rahul | Member |
| Aman | Member |
+-----+-----+
3 rows in set (0.002 sec)

```

Query 5: AGGREGATE (SUM)

Problem: Calculate total expense per group.

```
SELECT g.GroupName, SUM(e.TotalAmount) AS TotalExpense
```

```
FROM Expenses e
```

```
JOIN Groups g ON e.GroupID = g.GroupID
```

```
GROUP BY g.GroupName;
```

```

MariaDB [ExpenseSplitter]> SELECT g.GroupName, SUM(e.TotalAmount) AS TotalExpense
-> FROM Expenses e
-> JOIN Groups g ON e.GroupID = g.GroupID
-> GROUP BY g.GroupName;
+-----+-----+
| GroupName | TotalExpense |
+-----+-----+
| Flat Expenses | 3000.00 |
| Trip Group | 2000.00 |
+-----+-----+
2 rows in set (0.001 sec)

MariaDB [ExpenseSplitter]>

```

Query 6: AGGREGATE (COUNT)

Problem: Count members in each group.

```
SELECT g.GroupName, COUNT(gm.UserID) AS TotalMembers
```

```
FROM GroupMembers gm
```

```
JOIN Groups g ON gm.GroupID = g.GroupID
```

```
GROUP BY g.GroupName;
```

```

MariaDB [ExpenseSplitter]> SELECT g.GroupName, COUNT(gm.UserID) AS TotalMembers
-> FROM GroupMembers gm
-> JOIN Groups g ON gm.GroupID = g.GroupID
-> GROUP BY g.GroupName;
+-----+
| GroupName | TotalMembers |
+-----+
| Flat Expenses | 3 |
| Trip Group | 3 |
+-----+
2 rows in set (0.015 sec)

```

Query 7: SUBQUERY

Problem: Find user who paid highest expense.

SELECT Name

FROM Users

WHERE UserID IN (

SELECT PaidBy

FROM Expenses

WHERE TotalAmount = (SELECT MAX(TotalAmount) FROM Expenses)

);

```

MariaDB [ExpenseSplitter]> SELECT Name
-> FROM Users
-> WHERE UserID IN (
->     SELECT PaidBy
->     FROM Expenses
->     WHERE TotalAmount = (SELECT MAX(TotalAmount) FROM Expenses)
-> );
+-----+
| Name |
+-----+
| Rahul |
+-----+
1 row in set (0.002 sec)

```

Query 8: SUBQUERY

Problem: Find users in multiple groups.

SELECT Name

FROM Users

WHERE UserID IN (

SELECT UserID

FROM GroupMembers

GROUP BY UserID

HAVING COUNT(GroupID) > 1

);

```
MariaDB [ExpenseSplitter]> SELECT Name
-> FROM Users
-> WHERE UserID IN (
->     SELECT UserID
->     FROM GroupMembers
->     GROUP BY UserID
->     HAVING COUNT(GroupID) > 1
-> );
+-----+
| Name |
+-----+
| Rahul|
| Aman |
+-----+
2 rows in set (0.001 sec)
```

Query 9: INSERT

Problem: Insert a new user.

INSERT INTO Users (Name, Email, PasswordHash)

VALUES ('Rohit', 'rohit@gmail.com', 'pass123');

```
MariaDB [ExpenseSplitter]> INSERT INTO Users (Name, Email, PasswordHash)
-> VALUES ('Rohit', 'rohit@gmail.com', 'pass123');
Query OK, 1 row affected (0.003 sec)
```

Query 10: UPDATE

Problem: Update expense amount.

UPDATE Expenses

SET TotalAmount = 1500

WHERE ExpenseID = 1;

```
MariaDB [ExpenseSplitter]> UPDATE Expenses
-> SET TotalAmount = 1500
-> WHERE ExpenseID = 1;
Query OK, 1 row affected (0.004 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Query 11: DELETE

Problem: Delete a settlement.

DELETE FROM Settlements

WHERE SettlementID = 1;

```
MariaDB [ExpenseSplitter]> DELETE FROM Settlements
    -> WHERE SettlementID = 1;
Query OK, 1 row affected (0.004 sec)
```

Query 12: COMPLEX QUERY

Problem: Calculate balance of each user.

```
SELECT u.Name,
       SUM(es.PaidAmount) - SUM(es.OwedAmount) AS Balance
FROM ExpenseSplits es
JOIN Users u ON es.UserID = u.UserID
GROUP BY u.Name;
```

```
MariaDB [ExpenseSplitter]> SELECT u.Name,
    ->     SUM(es.PaidAmount) - SUM(es.OwedAmount) AS Balance
    -> FROM ExpenseSplits es
    -> JOIN Users u ON es.UserID = u.UserID
    -> GROUP BY u.Name;
+-----+-----+
| Name  | Balance |
+-----+-----+
| Aman  | -1666.67 |
| Priya | -1000.00 |
| Rahul | 2133.33  |
| shreya | 533.33  |
+-----+-----+
4 rows in set (0.018 sec)
```

5. Conclusion

This project demonstrates how DBMS can efficiently manage shared expenses. It ensures accuracy, transparency, and proper tracking of payments and balances.